

## Advanced SQL Analytics

In this step, I use more advanced SQL techniques to analyse the insurance data in a deeper and more efficient way. I use window functions to calculate things like rolling averages of claims over time and to rank high-risk customers without losing the detail at individual record level. I also use more complex joins to bring together different datasets, such as linking claims data with policy start and end dates, so I can properly analyse claims within the context of each policy's timeline.

### Customer Risk Ranking

#### Ranking customers by:

- **Their total number of claims**

```
SELECT
    cu.customerid,
    cu.name,
    cu.age,
    cu.location,
    cu.gender,
    COUNT(c.claimsid) AS claim_frequency,
    RANK() OVER (ORDER BY COUNT(c.claimsid) DESC) AS risk_rank
FROM customers cu
JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY
    cu.customerid,
    cu.name,
    cu.age,
    cu.location,
    cu.gender
ORDER BY claim_frequency DESC;
```

- **Their total claim cost**

```
SELECT
    cu.customerid,
    cu.name,
    cu.age,
    cu.location,
    cu.gender,
    SUM(c.amount) AS claim_total,
    RANK() OVER (ORDER BY SUM(c.amount) DESC) AS risk_rank
FROM customers cu
JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY
    cu.customerid,
    cu.name,
    cu.age,
    cu.location,
```

```

    cu.gender
having sum(c.amount)>0
ORDER BY claim_total DESC;

```

- **Their average claim severity**

```

SELECT
    cu.customerid,
    cu.name,
    cu.age,
    cu.location,
    cu.gender,
    COUNT(c.claimsid) AS number_of_claims,
    AVG(c.amount) AS avg_claim_amount,
    RANK() OVER (ORDER BY AVG(c.amount) DESC) AS risk_rank
FROM customers cu
JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY
    cu.customerid,
    cu.name,
    cu.age,
    cu.location,
    cu.gender
HAVING AVG(c.amount) > 0
ORDER BY avg_claim_amount DESC;

```

## Age-based risk ranking

### Rank age groups by:

- **Their average claim frequency**

```

SELECT
    cu.age,
    COUNT(c.claimsid) * 1.0 / COUNT(DISTINCT cu.customerid) AS average_claim_frequency,
    RANK() OVER (
        ORDER BY COUNT(c.claimsid) * 1.0 / COUNT(DISTINCT cu.customerid) DESC
    ) AS risk_rank
FROM customers cu
JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY cu.age
ORDER BY average_claim_frequency DESC;

```

- **Their total claims generated**

```

SELECT
    cu.age,
    SUM(c.amount) AS claim_total,
    RANK() OVER (
        ORDER BY SUM(c.amount) DESC
    ) AS risk_rank
FROM customers cu
JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c

```

```

    ON p.policyid = c.policyid
GROUP BY cu.age
ORDER BY claim_total DESC;

```

- **Their loss ratio (claims / premium)**

```

SELECT
    cu.age,
    COALESCE(SUM(c.amount), 0) AS claim_total,
    COALESCE(SUM(p.premium), 0) AS premium_total,
    COALESCE(SUM(c.amount), 0) / NULLIF(COALESCE(SUM(p.premium), 0), 0) AS loss_ratio,
    RANK() OVER (
        ORDER BY
            COALESCE(SUM(c.amount), 0) / NULLIF(COALESCE(SUM(p.premium), 0), 0) DESC
    ) AS risk_rank
FROM customers cu
JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY cu.age
ORDER BY risk_rank DESC;

```

## Coverage type risk ranking

### Rank coverage types by:

- **Their average claim frequency**

```

SELECT
    p.coveragetype,
    COUNT(c.claimsid) * 1.0 / COUNT(DISTINCT p.coveragetype) AS average_claim_frequency,
    RANK() OVER (
        ORDER BY COUNT(c.claimsid) * 1.0 / COUNT(DISTINCT p.coveragetype) DESC
    ) AS risk_rank
FROM policy p
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY p.coveragetype
ORDER BY average_claim_frequency DESC;

```

- **Their average claim severity**

```

SELECT
    p.coveragetype,
    sum(c.amount)/COUNT(c.claimsid) * 1.0 AS average_claim_severity,
    RANK() OVER (
        ORDER BY sum(c.amount)/COUNT(c.claimsid) * 1.0 DESC
    ) AS risk_rank
FROM policy p
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY p.coveragetype
ORDER BY average_claim_severity DESC;

```

- **Their loss ratio**

```

SELECT
    p.coveragetype,
    COALESCE(SUM(c.amount), 0) AS claim_total,
    COALESCE(SUM(p.premium), 0) AS premium_total,

```

```

        COALESCE(SUM(c.amount), 0) / NULLIF(COALESCE(SUM(p.premium), 0), 0) AS loss_ratio,
        RANK() OVER (
            ORDER BY COALESCE(SUM(c.amount), 0) / NULLIF(COALESCE(SUM(p.premium), 0), 0) DESC
        ) AS risk_rank
    FROM policy p
    LEFT JOIN claims c
        ON p.policyid = c.policyid
    GROUP BY p.coveragetype
    ORDER BY loss_ratio DESC;

```

## Time-based ranking

### Rank years by:

- **The total claims**

```

SELECT
    strftime('%Y', claimdate) AS claim_year,
    COUNT(*) AS total_claims,
    RANK() OVER (
        ORDER BY COUNT(*) DESC
    ) AS risk_rank
FROM claims
GROUP BY strftime('%Y', claimdate)
ORDER BY risk_rank;

```

- **The claim frequency(Average claims per customer per year)**

```

SELECT
    strftime('%Y', c.claimdate) AS claim_year,
    COUNT(c.claimsID) * 1.0 / COUNT(DISTINCT cu.customerID) AS claim_frequency,
    RANK() OVER (
        ORDER BY COUNT(c.claimsID) * 1.0 / COUNT(DISTINCT cu.customerID) DESC
    ) AS risk_rank
FROM customers cu
JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY strftime('%Y', c.claimdate)
ORDER BY risk_rank;

```

- **The total loss per year**

```

SELECT
    strftime('%Y', c.claimdate) AS claim_year,
    sum(c.amount) AS total_loss,
    RANK() OVER (
        ORDER BY sum(c.amount) DESC
    ) AS risk_rank
from policy p
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY strftime('%Y', c.claimdate)
ORDER BY risk_rank;

```

- **The Loss ratio per year**

```

SELECT
    strftime('%Y', p.startdate) AS policy_year,
    COALESCE(SUM(c.amount), 0) AS total_loss,
    SUM(p.premium) AS total_premium,
    COALESCE(SUM(c.amount), 0) * 1.0 / SUM(p.premium) AS loss_ratio,

```

```

RANK() OVER (
  ORDER BY COALESCE(SUM(c.amount), 0) * 1.0 / SUM(p.premium) DESC
) AS risk_rank
FROM policy p
LEFT JOIN claims c
  ON p.policyid = c.policyid
GROUP BY strftime('%Y', p.startdate)
ORDER BY risk_rank;

```

## Riskiest drivers

- **The top 10 customers by loss**

```

SELECT
  cu.customerid,
  cu.name,
  cu.age,
  cu.location,
  cu.gender,
  SUM(c.amount) AS claim_total,
  RANK() OVER (ORDER BY SUM(c.amount) DESC) AS risk_rank
FROM customers cu
JOIN policy p
  ON cu.customerid = p.customerid
LEFT JOIN claims c
  ON p.policyid = c.policyid
GROUP BY
  cu.customerid,
  cu.name,
  cu.age,
  cu.location,
  cu.gender
having sum(c.amount)>0
ORDER BY claim_total DESC
limit 10;

```

- **The top 10 riskiest age groups**

```

SELECT
  cu.age,
  SUM(c.amount) AS claim_total,
  RANK() OVER (
    ORDER BY SUM(c.amount) DESC
  ) AS risk_rank
FROM customers cu
JOIN policy p
  ON cu.customerid = p.customerid
LEFT JOIN claims c
  ON p.policyid = c.policyid
GROUP BY cu.age
ORDER BY claim_total DESC
limit 10;

```

## Relative risk ranking

- **The customers with above average risk**

```

WITH customer_risk AS (
  SELECT
    cu.customerid,

```

```

        SUM(c.amount) AS total_loss
FROM customers cu
LEFT JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY cu.customerid
),
portfolio AS (
    SELECT AVG(total_loss) AS avg_loss
    FROM customer_risk
)
SELECT
    cr.customerid,
    cr.total_loss,
    p.avg_loss,
    CASE
        WHEN cr.total_loss > p.avg_loss THEN 'Above Average Risk'
        ELSE 'Below Average Risk'
    END AS risk_flag
FROM customer_risk cr
CROSS JOIN portfolio p
ORDER BY cr.total_loss DESC;

```

- **The age groups above portfolio mean**

```

WITH age_risk AS (
    SELECT
        cu.age,
        SUM(c.amount) * 1.0 / COUNT(DISTINCT cu.customerid) AS avg_loss_per_customer
    FROM customers cu
    LEFT JOIN policy p
        ON cu.customerid = p.customerid
    LEFT JOIN claims c
        ON p.policyid = c.policyid
    GROUP BY cu.age
),
portfolio AS (
    SELECT AVG(avg_loss_per_customer) AS portfolio_avg
    FROM age_risk
)
SELECT
    ar.age,
    ar.avg_loss_per_customer,
    p.portfolio_avg,
    CASE
        WHEN ar.avg_loss_per_customer > p.portfolio_avg THEN 'Above Portfolio Risk'
        ELSE 'Below Portfolio Risk'
    END AS risk_flag
FROM age_risk ar
CROSS JOIN portfolio p
ORDER BY ar.avg_loss_per_customer DESC;

```

- **The coverage types above baseline loss**

```

WITH coverage_risk AS (
    SELECT
        p.coveragetype,
        SUM(c.amount) * 1.0 / COUNT(DISTINCT p.policyid) AS loss_per_policy
    FROM policy p
    LEFT JOIN claims c
        ON p.policyid = c.policyid

```

```

        GROUP BY p.coveragetype
    ),
    portfolio AS (
        SELECT AVG(loss_per_policy) AS baseline_loss
        FROM coverage_risk
    )
    SELECT
        cr.coveragetype,
        cr.loss_per_policy,
        p.baseline_loss,
        CASE
            WHEN cr.loss_per_policy > p.baseline_loss THEN 'Above Baseline Risk'
            ELSE 'Below Baseline Risk'
        END AS risk_flag
    FROM coverage_risk cr
    CROSS JOIN portfolio p
    ORDER BY cr.loss_per_policy DESC;

```

## Claim intensity ranking

- **total claims per policy duration**

```

SELECT
    p.lengthofpolicy,
    COUNT(c.claimsid) AS claim_frequency,
    RANK() OVER (ORDER BY COUNT(c.claimsid) DESC) AS risk_rank
FROM customers cu
JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY
    p.lengthofpolicy
ORDER BY risk_rank ASC;

```

- **Total amount per policy duration**

```

SELECT
    p.lengthofpolicy,
    sum(c.amount) AS claim_amount,
    RANK() OVER (ORDER BY sum(c.amount) DESC) AS risk_rank
FROM customers cu
JOIN policy p
    ON cu.customerid = p.customerid
LEFT JOIN claims c
    ON p.policyid = c.policyid
GROUP BY
    p.lengthofpolicy
ORDER BY risk_rank ASC;

```

## The rolling average

- SELECT
  - strftime('%Y', claimdate) AS claim\_year,
  - amount,
  - AVG(amount) OVER (
    - ORDER BY strftime('%Y', claimdate)
    - ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

```
) AS rolling_avg_claim  
FROM Claims  
group by claim_year  
ORDER BY claim_year;
```